

Setting Up Google Earth Engine with Kaggle Notebooks

A complete guide covering all registration steps, credential configuration, and code integration required to run Google Earth Engine (GEE) with Landsat imagery in a Kaggle notebook environment.

Overview

Running GEE inside a Kaggle notebook requires three separate registrations and one configuration step:

1. Google Earth Engine — register for noncommercial/education access
2. Google Cloud — create a service account with a JSON key
3. Kaggle Secrets — store the JSON key securely
4. Notebook code — authenticate and call the GEE API

Step 1 — Register for Google Earth Engine (Free Noncommercial Tier)

GEE is free for students, researchers, and educators under the noncommercial tier.[cite:9]

1.1 Go to the signup page

Navigate to: <https://earthengine.google.com/signup>

Click "Get Started" under *"See if you are eligible for noncommercial use"*.

1.2 Select your organization type

Choose: Academic or educational institution

1.3 Set your role

Select Student or Researcher and fill in your institution name.

1.4 Check eligibility

Click "Check eligibility" — a green confirmation should appear.

1.5 Select tier

Choose Community Tier (150 EECU-hours/month, no billing account required).[cite:3]

Tier	Monthly Quota	Billing Required	Best For
Community	150 EECU-hours	No	Students, coursework
Contributor	1,000 EECU-hours	Yes (free)	Heavy research

1.6 Describe your use case

Select "Classroom or education" or "Scientific research".

1.7 Link a Google Cloud Project

When prompted, select or confirm your existing Google Cloud Project (e.g., `my-first-project-123456`). If you do not have one, Google will create it automatically.[cite:7]

1.8 Enable the API

Click **Register**, then click **Enable** on the Earth Engine API prompt.[cite:7]

Step 2 — Create a Service Account and Download a JSON Key

Interactive browser authentication (`ee.Authenticate()`) does not work reliably in Kaggle's server environment. A service account with a JSON key is required instead.[cite:40]

2.1 Open the Service Accounts page

Go to: <https://console.cloud.google.com/iam-admin/serviceaccounts>

Select your project from the dropdown at the top.

2.2 Create a new service account

Click "+ Create Service Account" and fill in:

- **Name:** e.g., `gee-kaggle-client`
- **Description:** e.g., Earth Engine access for Kaggle notebooks

Click **Create and Continue**.

2.3 Assign a role

In the *Grant this service account access to project* section, add the role:

Earth Engine Resource Viewer (or Earth Engine Resource Writer if you need to export assets)

Click **Done**.

2.4 Register the service account with GEE

The service account email (e.g., `gee-kaggle-client@your-project.iam.gserviceaccount.com`) must be registered at:

https://signup.earthengine.google.com/#!/service_accounts

2.5 Download the JSON key

- Click on the newly created service account name
- Go to the **Keys** tab
- Click **Add Key** → **Create new key** → **JSON**
- A `.json` file will download automatically — keep it safe

The file looks like this:

```
{
  "type": "service_account",
  "project_id": "your-project-id",
  "private_key_id": "abc123...",
  "private_key": "-----BEGIN RSA PRIVATE KEY-----\n...",
  "client_email": "gee-kaggle-client@your-project.iam.gserviceaccount.com",
  "client_id": "...",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token"
}
```

Step 3 — Add the JSON Key to Kaggle Secrets

Kaggle Secrets allow storing sensitive credentials without exposing them in notebook code.[cite:27]

3.1 Open the Secrets panel

In the Kaggle notebook editor, click Add-ons (top menu) → Secrets.

3.2 Add a new secret

Click "Add a new secret" and fill in:

- **Label:** GEE_SERVICE_ACCOUNT_JSON (use this exact name in the code)
- **Value:** paste the **entire contents** of the downloaded .json file

3.3 Enable notebook access

In the Secrets panel, find your secret and toggle on "Notebook access" for the current notebook. Without this toggle, the secret cannot be read.[cite:23]

Step 4 — Write the Notebook Code

4.1 Cell 1 — Install dependencies (run once)

```
!pip install earthengine-api -q
```

folium is pre-installed in Kaggle and does not need to be installed separately.

4.2 Cell 2 — Authenticate and run

```
import os
os.environ["GOOGLE_MAPS_API_KEY"] = "" # prevents geemap/Colab timeout conflict

import ee
import json
import folium
from kaggle_secrets import UserSecretsClient

# Read credentials from Kaggle Secrets
secrets = UserSecretsClient()
key_json = secrets.get_secret("GEE_SERVICE_ACCOUNT_JSON")
key_data = json.loads(key_json)
```

```

# Authenticate with service account
credentials = ee.ServiceAccountCredentials(
    email=key_data["client_email"],
    key_data=key_json # must be a string, not a dict
)
ee.Initialize(credentials, project=key_data["project_id"])
print("GEE initialized:", key_data["project_id"])

# Filter Landsat 8 over Budapest
budapest = ee.Geometry.Point([19.0402, 47.4979])

landsat = (ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
    .filterBounds(budapest)
    .filterDate('2023-06-01', '2023-09-30')
    .filter(ee.Filter.lt('CLOUD_COVER', 10))
    .sort('CLOUD_COVER')
    .first()
)

vis_params = {
    'bands': ['SR_B4', 'SR_B3', 'SR_B2'],
    'min': 7000,
    'max': 20000,
    'gamma': 1.4
}

# Get tile URL from GEE
map_id = landsat.getMapId(vis_params)
tile_url = map_id['tile_fetcher'].url_format

# Render with Folium (works in Kaggle; geemap's ipyleaflet backend does not)
m = folium.Map(location=[47.4979, 19.0402], zoom_start=10)
folium.TileLayer(
    tiles=tile_url,
    attr='Google Earth Engine',
    name='Landsat 8 - Budapest',
    overlay=True,
    control=True
).add_to(m)
folium.LayerControl().add_to(m)
m

```

If the map does not render inline, save it as an HTML file:

```
m.save("budapest_landsat.html")
```

The file will appear in the **Output** tab on the right side of the Kaggle editor.

Why `folium` Instead of `geemap`?

The standard `geemap.Map` widget relies on `ipyleaflet`, which requires Jupyter widget extensions not available in Kaggle's notebook environment.[cite:51] Additionally, newer versions of `geemap` removed the `ee_folium` submodule.[cite:56] Using `folium` directly with GEE tile URLs is the most reliable approach in Kaggle.

Troubleshooting Reference

Error	Cause	Fix
<code>TimeoutException: Requesting secret GOOGLE_MAPS_API_KEY</code>	geemap tries to call <code>google.colab.userdata</code> inside Kaggle	<code>Set os.environ["GOOGLE_MAPS_API_KEY"] = ""</code> before importing geemap
No module named 'geemap.eefolium'	Module removed in newer geemap versions	Use <code>import folium</code> directly
<code>json.JSONDecodeError</code>	Malformed JSON in Kaggle Secrets	Re-paste the full <code>.json</code> file contents, including outer <code>{ }</code>
<code>EEException: Not signed up for Earth Engine</code>	Service account not registered with GEE	Register the service account email at <code>signup.earthengine.google.com/#!/service_accounts</code>
<code>AttributeError: 'NoneType' from get_secret()</code>	Secret label mismatch or toggle off	Check label name spelling and enable "Notebook access" toggle